МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет информационных технологий и программирования

ОТЧЁТ

по учебной ознакомительной практике

Тема: «Рефакторинг in-memory сервиса отложенных уведомлений»

Выполнили студенты группы М3102:

Шебалина Алиса
Ланских Глеб Константинович
Ручкин Олег Николаевич
Арвачев Михаил Дмитриевич

Санкт-Петербург, 2026

1. Постановка задачи

В ходе учебной практики выполнялся рефакторинг Notification Service — сервиса планирования отложенных уведомлений. Приложение хранит уведомления в оперативной памяти, позволяет создавать и отменять задания, просматривать их состояние и получать очередную порцию сообщений, готовых к отправке. В качестве каналов доставки предусмотрены email, push и SMS.

Отправкой сообщений занимается внешний worker. Он периодически обращается к GET /v1/due, передавая текущее время и ограничение на размер партии. Для такой модели важны не только скорость ответа, но и строгая последовательность выдачи, отсутствие повторов и предсказуемое поведение при массовых изменениях статусов.

Исходная версия проходила базовые проверки, однако на большой очереди проявлялись системные недостатки: полный обход всех запланированных идентификаторов при каждом запросе, повторная сортировка найденных записей, накопление уже отменённых задач и блокирование всех операций одним mutex. Дополнительно нарушались правила идемпотентности и порядка выдачи уведомлений.

Цель работы — устранить перечисленные проблемы за счёт переработки внутренних структур NotificationService, сохранив существующие HTTP-маршруты, сигнатуры публичных методов и формат взаимодействия с клиентом.

2. Исходная реализация и её ограничения

В baseline уведомления хранились в unordered_map по идентификатору, а последовательность schedule_ содержала все id в порядке добавления. Для получения готовых уведомлений метод due() проходил по schedule_, извлекал соответствующие записи, отсеивал будущие и завершённые задачи, затем сортировал весь найденный результат и лишь после этого применял limit.

Таким образом, ограничение limit=100 уменьшало размер ответа, но практически не ограничивало объём выполняемой работы. С ростом истории запросы GET /v1/due становились всё более дорогими, в том числе когда ни одно уведомление ещё не готово.

Состояния sent и cancelled дополнительно учитывались в отдельных множествах. После отмены идентификатор оставался в schedule_, а после завершения новая задача с тем же id могла ошибочно исключаться из выборки. Повторный POST с действующим id снова добавлял его в schedule_, создавая логические дубликаты.

Все методы использовали общий std::mutex. Даже независимые запросы чтения ожидали освобождения эксклюзивной блокировки, а длительный обход и сортировка due() задерживали создание и отмену задач.

3. Архитектурное решение

Основной принцип переработки — поддерживать порядок готовых заданий при изменении очереди, а не восстанавливать его заново при каждом запросе worker-а. Для этого введены индекс по id, отдельный упорядоченный индекс активных уведомлений и явная связь между ними.

Компонент	До рефакторинга	После рефакторинга
Хранилище по id	unordered_map<id, Notification>	unordered_map<id, NotificationRecord>
Готовая очередь	vector<string> schedule_	std::set<DueIndexEntry> due_index_
Терминальные статусы	sent_ и cancelled_ отдельно	status в NotificationRecord
Удаление из очереди	Логическая пометка	erase по сохранённому итератору
Параллельный доступ	std::mutex для всех операций	std::shared_mutex: read/write

3.1. Индекс готовности

В due_index_ хранится компактный ключ: время отправки send_at, приоритет priority, время создания created_at и идентификатор id. Стандартное упорядоченное множество std::set автоматически поддерживает нужный порядок. При добавлении pending-задачи создаётся запись в индексе, а соответствующий NotificationRecord сохраняет итератор на неё.

Такое решение не требует повторно сортировать уведомления при каждом GET /due. Worker начинает чтение с начала индекса, прекращает обход при достижении limit либо при встрече первого уведомления, запланированного позже now.

3.2. Единые правила сортировки и статусов

Порядок выдачи фиксирован четырьмя полями: сначала более ранний send_at, затем больший priority, затем более ранний created_at, после чего id в лексикографическом порядке. В baseline учитывались только send_at и id, поэтому задачи с одинаковым временем могли выдаваться не в соответствии с приоритетом.

```
send_at ↑   →   priority ↓   →   created_at ↑   →   id ↑
```

Для определения доступности уведомления используется его статус. Pending присутствует в due_index_; sent и cancelled хранятся в основном реестре для запросов GET, но отсутствуют в очереди готовности. Это устраняет расхождение между состоянием записи и дополнительными наборами маркеров.

4. Реализация обязательных задач

4.1. Ускорение GET /v1/due

Ранее горячий путь включал обход всей истории N идентификаторов и сортировку D подходящих уведомлений: O(N + D log D). После рефакторинга метод последовательно считывает первые K элементов подготовленного индекса; при среднем O(1) поиске записей в хеш-таблице затраты на выдачу ограниченной партии составляют O(K). Сам индекс поддерживается заранее за O(log A) на изменение, где A — число активных уведомлений.

4.2. Корректный приоритет уведомлений

Компаратор DueIndexEntryLess сравнивает четыре поля в установленной последовательности. Поэтому задача с более высоким приоритетом не уступает место низкоприоритетной только из-за лексикографического порядка id. Порядок остаётся одинаковым для повторных обращений при неизменном состоянии очереди.

4.3. Идемпотентность POST

Если новая запись поступает с id уже существующего pending-уведомления, метод add() оставляет действующую задачу без изменений. В частности, повторная передача того же id не дублирует запись в due_index_ и не перезаписывает её приоритет или время. Такой сценарий важен при повторных запросах клиента после сетевого таймаута.

4.4. Удаление отменённых и отправленных задач

При cancel() и markSent() соответствующий элемент удаляется из due_index_ по сохранённому итератору. Отменённые и отправленные уведомления перестают участвовать в обходе ready-очереди. В отличие от прежнего подхода, частота GET /due не зависит от количества ранее отменённых записей в schedule_. При этом историческая запись остаётся доступной по id — удаляется именно элемент горячего индекса, а не вся информация об уведомлении.

4.5. Повторное использование id после terminal state

Если прежняя запись находится в состоянии sent или cancelled, новый POST может заменить её новой pending-задачей с тем же внешним идентификатором. Старый элемент индекса, если он существовал, предварительно удаляется; затем создаётся актуальная запись. Это сохраняет ожидаемую семантику повторного планирования без ослабления идемпотентности действующих задач.

4.6. Снижение конкуренции между запросами

std::mutex заменён на std::shared_mutex. Методы get() и due() применяют разделяемую блокировку, поскольку только читают согласованные структуры, а add(), cancel() и markSent() используют исключительную блокировку. Одновременные запросы чтения больше не обязаны выстраиваться в одну очередь; при этом изменения карты и индекса остаются атомарными относительно других операций.

4.7. Быстрый ответ при отсутствии готовых задач

Индекс отсортирован по времени отправки прежде других полей. Если он пуст либо send_at первого элемента больше now, можно немедленно вернуть пустой результат без обхода будущих уведомлений. Проверка занимает O(1) времени после получения блокировки, даже если очередь содержит большое число отложенных заданий.

5. Сравнение сложности операций

Перенос части работы с частых запросов чтения на более редкие операции изменения очереди позволяет стабилизировать время ответа worker-a. Небольшое увеличение стоимости add(), cancel() и markSent() является оправданным обменом для read-heavy нагрузки.

Операция	Исходный подход	Новая реализация
GET /due (limit K)	O(N + D log D)	O(K) при среднем O(1) поиске по id
GET /due, no-ready	O(N)	O(1)
POST /notifications	O(1) в среднем, но копит id	O(log A) на обновление индекса
DELETE /notifications/{id}	Маркер, без очистки hot path	O(log A), исключение из индекса
POST /notifications/{id}/sent	Маркер, без очистки hot path	O(log A), исключение из индекса
GET /notifications/{id}	O(1) в среднем	O(1) в среднем

Обозначения: N — число записей в истории schedule_ до рефакторинга; D — число уведомлений, признанных готовыми при полном обходе; A — число активных pending-записей; K — количество запрошенных элементов (limit). Указанные оценки не включают ожидание блокировки и передачу HTTP-ответа.

6. Проверка корректности

После переработки включены пять тестов, которые в исходном проекте имели префикс DISABLED_. Они проверяют сортировку с учётом приоритета и created_at, отсутствие дубликатов при повторном POST, а также возможность повторного планирования после sent и cancelled. Вместе с исходными восемью сценариями тестовый набор насчитывает 13 проверок.

По материалам финальной презентации итоговый запуск GoogleTest завершился с результатом 13/13, без упавших тестов. Дополнительно основные сценарии были проверены отдельным локальным тестовым исполняемым файлом: 13 из 13 проверок прошли успешно, включая параллельные операции add(), due() и markSent().

Проверки покрывают три наиболее значимых инварианта: (1) pending-задача с данным id индексируется не более одного раза; (2) sent/cancelled не попадают в ответ due(); (3) при одновременном чтении и изменении данных сохраняется согласованность записей и элементов индекса.

Проект содержит возможность сборки с ThreadSanitizer через параметр ITMO_NOTIFICATION_TSAN. Это позволяет дополнительно контролировать гонки данных при воспроизведении многопоточных сценариев.

7. Производительность под нагрузкой

В презентации зафиксировано сравнение baseline и итоговой версии в сценарии scripts/load_test.py. Нагрузочные прогоны продолжались около 71 секунды и включали запросы на выдачу готовых уведомлений, создание новых задач, отмену и перевод в sent. Результаты сведены в таблицу.

Показатель	Baseline	После	Изменение
Суммарный RPS	1 334,6	7 686,1	×5,76
Успешный RPS	812,6	7 170,8	×8,82
GET /due, p50	6,76 мс	0,91 мс	в 7,43 раза ниже
GET /due, p95	117,04 мс	17,50 мс	в 6,69 раза ниже
GET /due, p99	158,59 мс	27,71 мс	в 5,72 раза ниже
Доля ошибок	39,1 %	6,7 %	в 5,84 раза ниже

Максимальный практический эффект связан с выдачей ограниченной партии: число запросов в секунду увеличилось более чем в пять раз, а задержка p99 у GET /due снизилась примерно с 159 до 28 мс. Существенно выросла и пропускная способность успешно обработанных запросов. При этом измеренный показатель p99 улучшился в 5,72 раза, то есть не достиг заявленного в исходном ТЗ порога 10×; остальные улучшения подтверждают устранение основного узкого места.

Снижение доли ошибок следует трактовать именно как изменение отношения ошибок к общему количеству запросов: общее число запросов в итоговом прогоне значительно выросло, тогда как абсолютное число ошибок изменилось лишь незначительно. На метрики также могут влиять окружение, параметры генератора и характер конкурирующей нагрузки.

8. Организация работы команды

Задачи распределялись по независимым проблемным областям, после чего изменения сводились в общую реализацию NotificationService. Распределение зафиксировано в итоговой презентации:

Участник	Основное направление
Ручкин Олег Николаевич	Оптимизация GET /due и корректный порядок выдачи (задачи 1–2)
Ланских Глеб Константинович	Идемпотентность POST и удаление отменённых задач из hot path (3–4)
Арвачев Михаил Дмитриевич	Повторное использование id и синхронизация shared_mutex (5–6)
Шебалина Алиса	Сценарий no-ready, тестирование и нагрузочные проверки (7)

9. Заключение

По итогам практики устранены все семь обязательных проблем исходного Notification Service без изменения публичного HTTP API. Вместо последовательного обхода и повторной сортировки введён индекс готовых задач; переходы между состояниями больше не оставляют мусор в горячем пути; повторные запросы корректно обрабатываются по id; чтение и запись разделены подходящими режимами синхронизации.

На приведённых в презентации измерениях суммарная производительность увеличилась в 5,76 раза, а задержка p99 выдачи готовых уведомлений сократилась в 5,72 раза. Регрессионный набор включает 13 тестов, в том числе пять ранее отключённых. В результате получена более предсказуемая реализация очереди, рассчитанная на частые запросы worker-а и большой объём запланированных уведомлений.

Опциональные расширения API, такие как claim, retry policy, rate limits, quiet hours и snapshot, в данном этапе не реализовывались: работа была сосредоточена на корректности и оптимизации основного сервиса.

Приложение А. Контрольные сценарии

Сценарий	Ожидаемое поведение
Повторный POST активного id	В due находится одна запись, свойства первой pending-задачи сохранены
Одинаковый send_at	Сначала больший priority, затем меньший created_at и id
Отмена и завершение	Запись доступна через GET, но не появляется в due
Новый POST после sent/cancelled	Под тем же id появляется новая pending-задача
Все записи имеют send_at > now	GET /due возвращает пустой массив без полного обхода очереди